

Project Proposal

Multimodal Deepfake Detection

A Cross-Modal Ablation Study of Spatial, Frequency, and Temporal Signals

Project Title and Team Members

Title: Multimodal Deepfake Detection

Team Members:

- Ayman Rabia — 017889495

Introduction

The rapid advancement of generative AI — particularly GANs and diffusion models — has made the creation of photorealistic fake video and audio increasingly accessible. Deepfakes pose serious threats to media integrity, public trust, and personal safety. While human observers struggle to distinguish synthetic from authentic media, machine learning models can be trained to detect subtle statistical artifacts left behind by the generation process.

This project builds a multimodal deepfake detection system that classifies video clips as real or fake by analyzing three complementary signal types: (1) spatial inconsistencies visible in the RGB frames, (2) frequency domain artifacts embedded in the pixel statistics, and (3) temporal inconsistencies across the frame sequence. By training models independently on each modality and then comparing against fusion architectures, this project produces both a deployable detector and a rigorous ablation study of each signal's contribution.

Problem Statement

The goal of this project is to build and compare a series of deepfake detection models of increasing sophistication, ranging from unimodal baselines to a full cross-attention fusion architecture, and to quantify the contribution of each input modality (spatial, frequency, temporal) to overall detection performance. A secondary goal is to evaluate cross-dataset generalization — training on one benchmark and testing on another — which is a known weakness in the existing literature.

Data Description

This project uses three publicly available deepfake detection benchmarks totaling approximately 62GB. FF++ serves as the primary training dataset; Celeb-DF v2 and DFDC Preview serve as independent cross-dataset generalization tests.

FaceForensics++ (FF++)

The primary training and evaluation dataset. FF++ contains 1,000 original videos and 4,000 manipulated videos generated by four different manipulation methods: Deepfakes, Face2Face, FaceSwap, and NeuralTextures. The compressed version (c23) will be used, approximately 50GB, which is the standard version used in published research. Video frames will be extracted at 5fps and face regions cropped using MediaPipe. Using FF++ as the training benchmark allows direct comparison with published baselines including Xception and EfficientNet variants.

Celeb-DF v2

Used exclusively for cross-dataset generalization evaluation (no training). Celeb-DF v2 contains 590 real and 5,639 deepfake videos of celebrities generated with a higher-quality synthesis pipeline than FF++. The higher visual fidelity of its fakes makes it a significantly harder test than FF++, and cross-dataset AUC on Celeb-DF v2 is a standard measure of real-world robustness in the literature. Approximately 2GB.

DFDC Preview (Facebook)

The DeepFake Detection Challenge Preview dataset, released by Facebook AI, is used as a second cross-dataset generalization test. Unlike FF++ and Celeb-DF, DFDC was collected in the wild with diverse subjects across ages, ethnicities, lighting conditions, and compression levels — making it the most challenging and realistic benchmark of the three. The preview subset (~10GB, ~5,000 clips) is used rather than the full 470GB dataset, as it was sampled to be representative of the full distribution and is the version commonly used in compute-constrained research settings.

Dataset summary:

Dataset	Real Videos	Fake Videos	Role	Size
FaceForensics++ (c23)	1,000	4,000	Training + primary eval	~50 GB
Celeb-DF v2	590	5,639	Cross-dataset test 1	~2 GB
DFDC Preview	~1,000	~4,000	Cross-dataset test 2	~10 GB

Methodology

All models will be implemented in PyTorch. Preprocessing will be handled locally; training will run on Google Colab A100 GPU with High-RAM enabled. The project proceeds in three phases:

Phase 1 — Unimodal Baselines

Three independent models are trained, each receiving only one type of input signal:

- **Spatial CNN:** A custom ResNet-style CNN trained on raw RGB face crops (224x224). Learns to detect visual artifacts such as blurred hairlines, asymmetric facial geometry, and lighting inconsistencies.
- **Frequency CNN:** The same CNN architecture trained on FFT magnitude spectra of the same face crops. Learns to detect GAN grid artifacts and diffusion model fingerprints invisible to the human eye.
- **Temporal CNN+LSTM:** 16 frames sampled per clip, each encoded by a shared CNN, then passed through an LSTM to capture motion inconsistencies across time — such as unnatural blinking patterns or missing micro-expressions.

Phase 2 — Fusion Architectures

Three fusion strategies are compared:

- **Late Fusion:** The three baseline models are trained independently; their softmax outputs are averaged at inference time. A simple but often competitive baseline.
- **Concatenation Fusion:** All three stream embeddings are concatenated into a single 768-dim vector and passed through an MLP classifier. Trained end-to-end jointly.
- **Cross-Attention Fusion:** The audio/frequency embedding attends over video frame features at each timestep using a multi-head attention mechanism. The most expressive architecture and the primary research contribution.

Phase 3 — Pretrained Backbone Comparison

Xception (the published SOTA baseline for FF++) is evaluated under identical conditions to benchmark the from-scratch models. A version with EfficientNetV2 backbone replacing the custom CNN is also evaluated to isolate the contribution of pretraining vs. architecture.

Objectives

1. Build and validate a preprocessing pipeline for frame extraction, face cropping, and FFT computation
2. Train all unimodal baseline models and record per-class accuracy, precision, recall, and F1
3. Implement and train all three fusion architectures
4. Evaluate all models on FF++ test set, Celeb-DF v2, and DFDC Preview (cross-dataset generalization)
5. Produce an ablation table showing the marginal contribution of each modality
6. Build a FastAPI backend and web frontend with Grad-CAM visualization for demo purposes

Evaluation Metrics

The following metrics will be reported for all models on both datasets:

- Binary accuracy (real vs. fake)
- AUC-ROC — primary metric for comparison with published results
- Per-manipulation-method accuracy on FF++ (Deepfakes, Face2Face, FaceSwap, NeuralTextures)
- Cross-dataset AUC on Celeb-DF v2 — generalization to higher-quality fakes
- Cross-dataset AUC on DFDC Preview — generalization to real-world, in-the-wild conditions
- Grad-CAM activation maps on spatial stream — qualitative explainability

AUC-ROC is prioritized over raw accuracy because the dataset has a 4:1 fake-to-real imbalance and because it is the standard metric in the deepfake detection literature, enabling direct comparison with published baselines. Cross-dataset AUC on both Celeb-DF v2 and DFDC Preview is reported separately to distinguish generalization to high-quality fakes from generalization to real-world in-the-wild conditions.

Compute Plan

Task	Environment	Estimated RAM
Dataset download + frame extraction	Local	~4 GB
FFT preprocessing + face crop caching	Local	~6 GB

Architecture prototyping	Local / Colab T4	~8 GB
Spatial + Frequency CNN training	Colab T4	~10 GB VRAM
Temporal CNN+LSTM training	Colab A100	~18 GB VRAM
Full fusion model training	Colab A100 + High-RAM	~30 GB VRAM
Inference + demo serving	Local	~6 GB

All training runs will checkpoint to Google Drive every epoch. The preprocessing pipeline runs entirely locally to avoid Colab session limits on data-heavy operations.

Implementation Steps

The following steps define the full implementation sequence in order. Each step must be completed and verified before the next begins. This document is intended to serve as a reference for both the developer and any AI coding assistant (such as Claude Code) supporting implementation.

Step 1 — Repository and Project Structure

- Create GitHub repository: deepfake-detection
- Create directories: data/raw, data/processed/frames, data/processed/faces, data/processed/fft, data/processed/sequences, data/splits, preprocessing/, models/spatial, models/frequency, models/temporal, models/fusion, notebooks/, api/, frontend/, results/
- Add .gitignore that excludes the entire data/ directory, __pycache__, .DS_Store, .env, and notebook checkpoints
- Add requirements.txt with: opencv-python, mediapipe, numpy, pandas, matplotlib, scikit-learn, torch, torchvision, fastapi, uvicorn, tqdm
- Add placeholder README.md

Step 2 — Dataset Acquisition

- FaceForensics++: Submit access request form at github.com/ondyari/FaceForensics to receive the download script. Run with flags for all four manipulation types at compression level c23. Expected size: ~50GB. Run overnight.
- Celeb-DF v2: Download via Google Drive link at github.com/yuezunli/celeb-deepfakeforensics. Expected size: ~2GB.

- DFDC Preview: Download via Kaggle CLI after accepting competition terms at kaggle.com/competitions/deepfake-detection-challenge. Expected size: ~10GB.
- After download: verify file counts match documented dataset sizes before proceeding.

Step 3 — Preprocessing Pipeline (Local)

All preprocessing runs locally and outputs cached files to disk. Each script must support re-runs safely by skipping already-processed files.

- `extract_frames.py` — Takes raw video files, samples every 5th frame, saves as JPEG to `data/processed/frames/`. Use OpenCV. Preserve source subfolder structure.
- `crop_faces.py` — Detects face using MediaPipe, adds 20% padding, crops and resizes to 224x224, saves to `data/processed/faces/`. Skip and log frames where no face is detected.
- `compute_fft.py` — Converts face crop to grayscale, applies `np.fft.fft2()` + `np.fft.fftshift()`, takes log magnitude with `np.log1p(np.abs())`, normalizes to 0-255, saves as JPEG to `data/processed/fft/`. Output filename must match input filename exactly.
- `build_dataset.py` — Infers label and source from folder names, splits at video level 70/15/15 stratified by label, outputs `train.csv`, `val.csv`, `test.csv` with columns: `frame_path`, `fft_path`, `label`, `source`, `manipulation_type`, `split`.
- `verify_dataset.py` — Prints summary table, checks all file paths exist on disk, renders sample grids of real vs fake faces and FFT images, warns if any split exceeds 60/40 class imbalance.

Step 4 — Unimodal Model Training (Colab A100)

Upload preprocessed data/ to Google Drive before this step. All notebooks checkpoint to Drive every epoch.

- Notebook 02: Spatial CNN — Custom ResNet-style CNN on 224x224 RGB face crops. Train on FF++ train split. Record accuracy, AUC, F1 per manipulation type.
- Notebook 03: Frequency CNN — Same architecture on 224x224 FFT magnitude images. Same training protocol.
- Notebook 04: Temporal CNN+LSTM — 16 frames per clip, shared CNN encoder, LSTM over sequence. Batch size 8 due to memory.
- Log all results to `results/` as JSON files.

Step 5 — Fusion Model Training (Colab A100)

- Late Fusion: Average softmax outputs of three trained unimodal models. No additional training required. Evaluate on all three datasets.
- Concatenation Fusion: Concatenate all three stream embeddings to 768-dim vector, pass through MLP. Train end-to-end jointly.
- Cross-Attention Fusion: Multi-head attention across all three streams. Train end-to-end. This is the primary research contribution.

Step 6 — Pretrained Backbone Comparison (Colab A100)

- Fine-tune Xception on FF++ train split. Evaluate on all three datasets. Direct SOTA comparison.
- Fine-tune EfficientNetV2 backbone. Compare against from-scratch CNN to isolate the value of ImageNet pretraining.
- Record all results in ablation table in results/.

Step 7 — Cross-Dataset Evaluation

Run all models against Celeb-DF v2 and DFDC Preview without any additional fine-tuning. Record AUC per model per dataset and compile final ablation table.

Step 8 — Demo Application (Local)

- FastAPI backend (api/): endpoints for each model type. Accept video upload, run preprocessing on the fly, return predictions.
- Frontend (frontend/): HTML/JS interface. Upload video, show per-frame predictions, side-by-side confidence bar charts per model, Grad-CAM heatmap on spatial stream.
- Export models to ONNX format for faster CPU inference if needed for local serving.

Step 9 — Write-up and Portfolio Polish

- Fill in results table in README.md with final numbers
- Write ablation analysis section explaining each modality's contribution
- Add Grad-CAM visualization examples to README
- Record short screen capture demo video for GitHub README
- Clean up all notebooks, add markdown explanations to each cell

Compute Environment and Constraints

This section documents the full hardware and software environment so that any developer or AI coding assistant has complete context before writing or running code.

Local Machine (macOS — Preprocessing and Demo Only)

- RAM: 16GB unified memory. All preprocessing scripts must use lazy loading and process files one at a time. Never load the full dataset into memory.
- GPU: None available locally. PyTorch will run on CPU only. All code must check `torch.cuda.is_available()` and fall back to CPU gracefully. Never hardcode `.cuda()` calls.
- Architecture: Apple Silicon (ARM). Use ARM-compatible pip wheels. MediaPipe and OpenCV both have ARM-compatible releases.
- Storage: Preprocessed data stored locally during preprocessing, then uploaded to Google Drive before Colab training. The `data/` directory is always gitignored.
- Python version: 3.10 or later. Use a virtual environment (venv or conda) for all dependencies.
- Role: Dataset download, all preprocessing scripts, final demo serving only. No training happens here.

Google Colab Pro (Primary Training Environment)

- T4 GPU (15GB VRAM, ~12GB system RAM): Used for spatial CNN, frequency CNN, and prototyping. Session limit ~12 hours. Checkpoint every epoch.
- A100 GPU (40GB VRAM): Used for temporal CNN+LSTM, all fusion models, and cross-dataset evaluation. Enable High-RAM mode for ~52GB system RAM. Session limit ~24 hours on Pro. Checkpoint every epoch.

Critical constraints all Colab code must respect:

- Sessions disconnect without warning. Every training script must save a checkpoint at the end of every epoch to Google Drive. Training must be resumable from the last checkpoint.
- Data lives on Google Drive. Mount Drive at the start of every notebook with `drive.mount('/content/drive')`. All data paths must be relative to the Drive mount point.
- Colab VMs reset between sessions. All pip installs must be at the top of every notebook — do not assume packages persist between sessions.
- Use mixed precision training (`torch.cuda.amp`) on both T4 and A100 to reduce VRAM usage and speed up training.

- Batch sizes: Spatial/Frequency CNN: 64 on A100, 32 on T4. Temporal CNN+LSTM: 16 on A100, 8 on T4. Fusion models: 16 on A100 only.

Environment Summary

Task	Environment	GPU	RAM	Notes
Dataset download	Local	None	16 GB	Run overnight
Preprocessing scripts	Local	None	16 GB	One file at a time
Architecture prototyping	Local or T4	None / T4	16 GB / 12 GB	CPU fallback required
Spatial + Frequency CNN	Colab T4	T4 15GB	12 GB	Checkpoint every epoch
Temporal CNN+LSTM	Colab A100	A100 40GB	52 GB	High-RAM on
All fusion models	Colab A100	A100 40GB	52 GB	High-RAM on
Cross-dataset evaluation	Colab A100	A100 40GB	52 GB	No fine-tuning
Demo serving	Local	None	16 GB	ONNX export if needed

Key Libraries

- PyTorch 2.x + torchvision — all model code
- OpenCV (cv2) — video frame extraction
- MediaPipe — face detection and cropping
- NumPy — FFT computation
- Pandas — dataset CSV management
- scikit-learn — train/val/test splitting, evaluation metrics
- Matplotlib — verification visualizations
- FastAPI + Uvicorn — demo backend API
- tqdm — progress bars on all long-running scripts

Related Work

Rossler et al. (2019) introduced FaceForensics++ and established Xception as a strong baseline, achieving high accuracy on compressed video but noting significant degradation under cross-dataset evaluation. Li et al. (2020) showed that frequency domain features, particularly GAN grid artifacts visible in FFT spectra, provide complementary signal to spatial features. Subsequent work on temporal modeling demonstrated that biological motion cues such as blinking and head movement provide additional discriminative signal not present in single-frame analysis. The DFDC challenge (Dolhansky et al., 2020) further highlighted that models trained on controlled lab datasets fail to generalize to in-the-wild conditions, motivating the use of DFDC Preview as a second generalization benchmark alongside Celeb-DF v2. This project directly builds on these lines of work by combining all three signal types in a unified architecture and evaluating cross-dataset robustness across two independent benchmarks.

Timeline

Week	Milestone
1–2	Dataset download, frame extraction pipeline, FFT preprocessing, MediaPipe face cropping
3–4	Spatial CNN and Frequency CNN baselines trained and evaluated on FF++
5	Temporal CNN+LSTM baseline trained; late fusion baseline evaluated
6–7	Concatenation and cross-attention fusion models trained and compared
8	Xception + EfficientNetV2 backbone comparison; cross-dataset Celeb-DF evaluation
9	FastAPI backend + Grad-CAM frontend demo
10	Final write-up, README, GitHub polish, ablation table finalized

Conclusion

This project proposes a structured, rigorous approach to multimodal deepfake detection that goes beyond single-modality classification. By training from scratch on spatial, frequency, and temporal signals and systematically comparing fusion strategies, the project produces both a technically grounded ablation study and a deployable detection system with visual explainability. The cross-dataset evaluation across two independent benchmarks — Celeb-DF v2 and DFDC Preview — directly addresses the generalization gap that limits most published work in this area. The outcome will serve as a strong portfolio artifact demonstrating end-to-end ML engineering, original architecture design, and applied research methodology.

References

- Rossler, A. et al. FaceForensics++: Learning to Detect Manipulated Facial Images. ICCV 2019.
- Li, Y. et al. Frequency-aware Discriminative Feature Learning Supervised by Single-Center Loss for Face Forgery Detection. CVPR 2021.
- Celeb-DF: A Large-scale Challenging Dataset for DeepFake Forensics. CVPR 2020.
- Dolhansky, B. et al. The DeepFake Detection Challenge (DFDC) Dataset. arXiv 2020.
- FaceForensics++ Dataset: <https://github.com/ondyari/FaceForensics>
- Celeb-DF v2 Dataset: <https://github.com/yuezunli/celeb-deepfakeforensics>
- DFDC Preview Dataset: <https://ai.facebook.com/datasets/dfdc/>